---

1. Encapsulation

Concept

Encapsulation means wrapping data (variables) and behavior (methods) together into a single unit (class).

It also controls **how the data is accessed**.

In C#, encapsulation is achieved using:

- **Classes**
- **Access Modifiers** (private, public, protected)
- **Properties**

Simple Definition

Encapsulation = **Binding data and methods together in a class and controlling access to them.**

Why Encapsulation is Needed

Without encapsulation:

- Anyone can change data
- Data becomes unsafe
- Bugs increase

Encapsulation ensures:

- Data protection
- Controlled access
- Better maintainability

Real-Life Example

Think about a **Bank ATM Machine**.

You interact with:

- Withdraw
- Deposit
- Check balance

But you **cannot directly access the bank database**.

So the **data is hidden**, and operations are controlled.

Practical C# Example

Without Encapsulation (Bad Practice)

```
class Patient
{
    public string patientName;
    public int age;
}
```

Usage

```
Patient p = new Patient();
p.patientName = "John";
p.age = -10;   // invalid data but allowed
```

Problem:

- Anyone can assign **invalid values**

With Encapsulation (Correct Way)

```
class Patient
{
    private int age;

    public void SetAge(int value)
```

```
    {
        if(value > 0)
            age = value;
    }

    public int GetAge()
    {
        return age;
    }
}
```

Usage

```
Patient p = new Patient();

p.SetAge(25);
Console.WriteLine(p.GetAge());
```

Now:

- Age cannot be negative
- Data is controlled

Encapsulation Using Properties (Recommended)

```
class Patient
{
    private int age;

    public int Age
    {
```

```
        get { return age; }
        set
        {
            if(value > 0)
                age = value;
        }
    }
}
```

Usage

```
Patient p = new Patient();


p.Age = 30;
Console.WriteLine(p.Age);
```

---

2. Data Hiding

Concept

Data Hiding means restricting direct access to class data.

It is achieved using **access modifiers**.

Example:

- private
- protected
- internal

Simple Definition

Data Hiding = **Protecting class data from direct access.**

Real-Life Example

Think of a **Medicine Capsule** 💊

Inside:

- chemicals
- formulas

But you **cannot see or modify them**.

You only consume the capsule.

Same idea in programming.

Practical C# Example

```
class Account
{
    private double balance;

    public void Deposit(double amount)
    {
        balance += amount;
    }

    public double GetBalance()
    {
        return balance;
    }
}
```

Usage

```
Account acc = new Account();
```

```
acc.Deposit(5000);


Console.WriteLine(acc.GetBalance());
```

Here:

- balance is **hidden**
- It cannot be accessed directly

Invalid usage (not allowed)

```
acc.balance = 10000;  // ERROR
```

Key Point

Concept	Meaning
Encapsulation	Wrapping data and methods together
Data Hiding	Restricting access to internal data

Data hiding is part of encapsulation.

---

3. Abstraction

Concept

Abstraction means showing only essential features and hiding internal implementation details.

User knows:

- What the system does

User does NOT know:

- How it does it

Real-Life Example

Think of a **Car** 🚙

You only know:

- Start car
- Accelerate
- Brake

You **do not need to know**:

- Engine combustion
- Fuel injection system

That complexity is hidden.

In C#

Abstraction is implemented using:

1. **Abstract Classes**
2. **Interfaces**

Practical Demo Using Abstract Class

Step 1: Create Abstract Class

```
abstract class Payment
{
    public abstract void Pay();
}
```

This means:

- Payment type is defined
- Implementation not defined

Step 2: Implement Child Classes

```
class CreditCardPayment : Payment
{
    public override void Pay()
    {
        Console.WriteLine("Payment done using Credit Card");
    }
}

class UpiPayment : Payment
{
    public override void Pay()
    {
        Console.WriteLine("Payment done using UPI");
    }
}
```

Step 3: Use the Abstraction

```
Payment p;

p = new CreditCardPayment();
p.Pay();

p = new UpiPayment();
p.Pay();
```

Output

Payment done using Credit Card

Payment done using UPI

User only knows **Pay()**, not how it works.

Abstraction Using Interface

```
interface INotification
{
    void Send();
}
```

Implementation

```
class EmailNotification : INotification
{
    public void Send()
    {
        Console.WriteLine("Email Sent");
    }
}

class SmsNotification : INotification
{
    public void Send()
    {
        Console.WriteLine("SMS Sent");
    }
}
```

Usage

```
INotification n = new EmailNotification();
```

```
n.Send();
```

Difference Between Encapsulation, Data Hiding, Abstraction

Concept	Purpose	How it is Achieved
Encapsulation	Bind data + methods	Class
Data Hiding	Protect data	Access modifiers
Abstraction	Hide implementation	Abstract class / Interface

💡 **Simple Interview Definition**

Encapsulation

Binding data and methods together into a single unit and controlling access.

Data Hiding

Restricting direct access to class data using access modifiers.

Abstraction

Showing only necessary features while hiding implementation details.